

System Design Fundamentals

A simple, memorable revision guide for FAANG and MAANG interviews

What this guide optimizes for

Clear definitions, realistic examples, interview-ready trade-offs, and a full Prime-like recommendation system built from scratch. Every number in the case study is an illustrative interview assumption, not a claim about Amazon's internal systems.

How to remember the subject

Understand the request. Estimate the load. Draw the path. Store the state. Plan for failure. Explain the trade-off.

Prepared as a practical revision handbook

Contents

Use the headings as a study map. The PDF outline is also available in the bookmarks panel.

Part 1 - The mental model	5
1. What the interviewer is testing	5
2. The interview conversation in seven moves	5
Part 2 - Core vocabulary in plain English	7
3. Functional and non-functional requirements	7
4. Strong versus eventual consistency	7
Part 3 - The building blocks	8
5. Client, edge, and API gateway	8
6. Load balancer and stateless services	8
7. Cache	8
8. Database choice	9
Part 4 - Scale, data movement, and protection	10
9. Replication	10
10. Sharding or partitioning	10
11. Queue, stream, and asynchronous work	10
12. CDN and object storage	10
13. Rate limiting and backpressure	11
Part 5 - Data contracts, reliability, and observability	12
14. API design	12
15. Idempotency, retries, and exactly-once myths	12
16. Observability	12
17. Security basics	12
Part 6 - Capacity estimation without fear	13
18. The four estimates to practice	13
19. Worked example: viewer events	13
20. Latency budgets	13
21. Common estimation mistakes	13
Part 7 - Trade-offs and mini designs	14
22. Trade-off pairs worth remembering	14
23. Mini design: URL shortener	14
24. Mini design: notification service	14
25. Failure checklist	14
Part 8 - Capstone: a Prime-like recommendation system	15
26. The product problem	15
27. Why do we need a recommendation system?	15
28. Define success before choosing a model	15
Part 9 - Recommendation system requirements and assumptions	16

29. Functional requirements	16
30. Non-functional requirements	16
31. Non-goals for the first version	16
Part 10 - High-level architecture	17
32. The big picture	17
33. Main components	17
Part 11 - Request-time flow: from open screen to ranked rows	18
34. Step-by-step serving path	18
35. Example API	18
Part 12 - Event collection and data flow	19
36. Events to collect	19
37. Event design details	19
Part 13 - Candidate generation	20
38. Candidate sources	20
39. Combining candidates	20
40. Candidate store	20
41. Cold-start cases	20
Part 14 - Ranking and re-ranking	21
42. Ranking objective	21
43. Features	21
44. Re-ranking rules	21
Part 15 - Data science, AI, and ML components	22
45. What the data science team owns	22
46. A practical model ladder	22
Part 16 - Training pipeline and feature store	23
47. Training pipeline	23
48. Feature store in simple terms	23
49. Model serving choices	23
Part 17 - Online serving, reliability, and cost	24
50. Latency budget example	24
51. Failure modes and fallbacks	24
52. Cost controls	24
Part 18 - Evaluation, experimentation, and feedback loops	25
53. Offline metrics	25
54. Online A/B testing	25
55. Feedback loop risks	25
Part 19 - Privacy, safety, and governance	26
56. Protecting viewer data	26
57. Content and policy layer	26
58. Governance checklist	26

Part 20 - A polished interview answer you can say aloud	27
59. Opening	27
60. Requirements and scale	27
61. Architecture	27
62. Reliability	27
63. Trade-off close	27
Part 21 - Final one-page memory sheet	28
64. The universal system-design checklist	28
65. Recommendation-system checklist	28

Part 1 - The mental model

Start with the few ideas that make every design easier to reason about.

System design is the skill of deciding how software components work together when many users, much data, and failures are involved. The interviewer is not looking for one magical architecture. They are looking for a clear thought process and sensible trade-offs.

Memory hook: R-E-A-C-H

Requirements -> Estimate -> API and data -> Components -> Hardening for scale, failure, security, and observability.

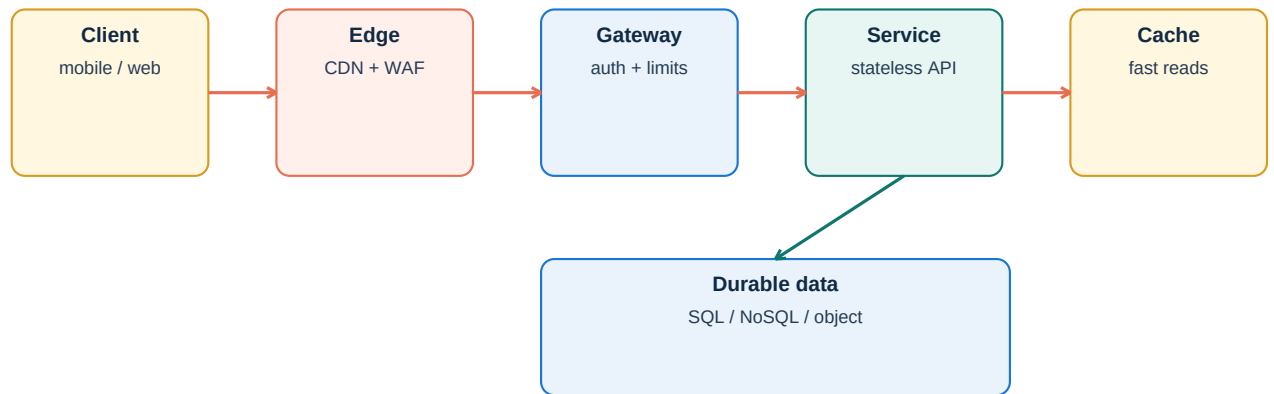
1. What the interviewer is testing

Skill	Plain meaning	What good sounds like
Clarification	You find the real problem before drawing boxes.	"Are we optimizing for low latency, high freshness, or both?"
Decomposition	You split a large problem into smaller responsibilities.	"The API, recommendation logic, and event pipeline can evolve separately."
Scale reasoning	You connect user traffic to storage, compute, and network load.	"At this traffic, a single database becomes a bottleneck; we need replicas or partitions."
Trade-offs	You explain what gets better and what becomes harder.	"Caching lowers latency and database load, but freshness becomes a design choice."
Failure thinking	You decide what happens when a dependency is slow or down.	"If ranking times out, return a cached popular list."

2. The interview conversation in seven moves

- **Clarify:** users, core actions, read/write shape, region, privacy, and success metrics.
- **Estimate:** daily active users, requests per second, storage, bandwidth, and peak traffic.
- **Define the contract:** important APIs and the data each request needs.
- **Draw the simplest working design:** client -> edge -> service -> cache/database.
- **Scale it:** stateless instances, load balancing, caching, partitioning, replicas, queues.
- **Harden it:** timeouts, retries, idempotency, rate limits, monitoring, security, and fallbacks.
- **Close the loop:** summarize trade-offs and ask what the interviewer wants to explore next.

A request usually moves left to right; durable state sits below the stateless layer.



Interview sentence

"I will start with the smallest design that meets the stated requirements, then scale the bottlenecks one by one. I will call out each trade-off instead of adding infrastructure just because it is popular."

Part 2 - Core vocabulary in plain English

Learn the meaning first; memorize the name second.

3. Functional and non-functional requirements

Term	Simple definition	Example
Functional requirement	What the system must do.	A viewer can request a home page with recommended titles.
Non-functional requirement	How well the system must do it.	The home page loads in under 200 ms at the 99th percentile.
Latency	Time for one request to get a response.	Search results return in 80 ms.
Throughput	How many requests or events the system handles per second.	The event collector accepts 50,000 watch events per second.
Availability	How often the system is usable.	99.99% availability means about 52 minutes of downtime per year.
Reliability	How consistently the system behaves correctly over time.	Payments do not get charged twice.
Durability	Whether stored data survives failures.	A completed purchase is still present after a machine dies.
Consistency	What readers are allowed to see after a write.	A just-created playlist appears immediately, or may appear a few seconds later.
Scalability	How the system handles more users or data.	Adding servers increases capacity without redesigning everything.
SLO / SLA	SLO is an internal target; SLA is a customer promise or contract.	SLO: 99.9% success. SLA: credits if the promise is missed.

Memory hook: fast, many, always, correct, safe

Latency = fast. Throughput = many. Availability = always reachable. Reliability = correct behavior. Durability = data survives. Consistency = what version readers see.

4. Strong versus eventual consistency

Strong consistency: after a write succeeds, every later read sees the newest value. It is easy to reason about, but may cost more latency or availability during a network problem.

Eventual consistency: replicas may temporarily disagree, but if writes stop, they converge. It is often a good fit for feeds, counters, recommendations, and analytics where a small delay is acceptable.

Example	Reasonable choice	Why
Bank balance	Strong consistency	A stale balance can cause financial harm.
Like count	Eventual consistency	Being off by a few seconds is usually acceptable.
Video recommendation row	Eventual consistency	Freshness matters, but a cached list is still useful.

Part 3 - The building blocks

These are the boxes you reuse across most interview questions.

5. Client, edge, and API gateway

The **client** is the web, mobile, TV, or service that starts a request. The **edge layer** is close to the user and may provide CDN caching, TLS termination, DDoS protection, and a web application firewall. The **API gateway** is the front door for backend APIs: it authenticates, routes, rate-limits, and adds request metadata.

Example

When a viewer opens the Prime-like home page, the request first reaches the nearest edge location. The gateway checks the token and sends the request to the recommendation service.

6. Load balancer and stateless services

A **load balancer** spreads requests across healthy service instances. A **stateless service** does not keep user session state only in its own memory. Any instance can handle the next request, which makes horizontal scaling and failover much easier.

Example: If three recommendation API instances each handle 1,000 requests per second, the load balancer can send roughly 3,000 requests per second across them. If one dies, traffic moves to the remaining two.

Memory hook: keep compute replaceable

Store important state in a database, cache, or session store. Keep application instances easy to add, remove, and replace.

7. Cache

A cache stores frequently used data closer to the application so the system can answer faster and reduce load on the database. Common choices are an in-memory key-value store or an edge cache.

Pattern	Meaning	Example
Cache-aside	Application checks cache; on a miss, reads the database and fills the cache.	Read user preferences; cache the result for 5 minutes.
Write-through	Application writes cache and database together.	Useful when the cached value must be immediately available.
TTL	Time to live; data expires after a chosen period.	Trending titles expire every 60 seconds.
Invalidation	Remove or refresh stale data after a write.	Delete the profile cache after a preference change.

Classic warning: cache invalidation is hard because you must decide when old data is no longer acceptable. Cache only what you can tolerate serving stale.

8. Database choice

Data shape	Good first choice	Use it when...
Relationships and transactions	Relational database	Orders, payments, permissions, and data with joins.
Simple key lookup at very high scale	Key-value / wide-column store	You know the access pattern and need predictable horizontal scale.
Flexible document	Document database	A record has nested fields that change often.
Edges and relationships	Graph database	Queries follow many relationships, such as social connections.
Large files	Object storage	Videos, images, backups, and model artifacts.
Search and ranking	Search index	Text search, filters, autocomplete, and relevance ranking.

Interview rule

Start from the access pattern, not from the database brand. Say: "We need to read X by key, write Y events, and query Z by time; that is why I choose this storage model."

Part 4 - Scale, data movement, and protection

Now turn a single-machine design into a production-shaped system.

9. Replication

Replication keeps copies of data on multiple machines. A **leader** accepts writes and followers copy them. Read replicas can spread read traffic. Replication improves availability and read capacity, but introduces lag and conflict questions.

Example: A catalog database has one leader for title updates and three read replicas for home-page reads. If a replica is 2 seconds behind, the system may temporarily show an old title description.

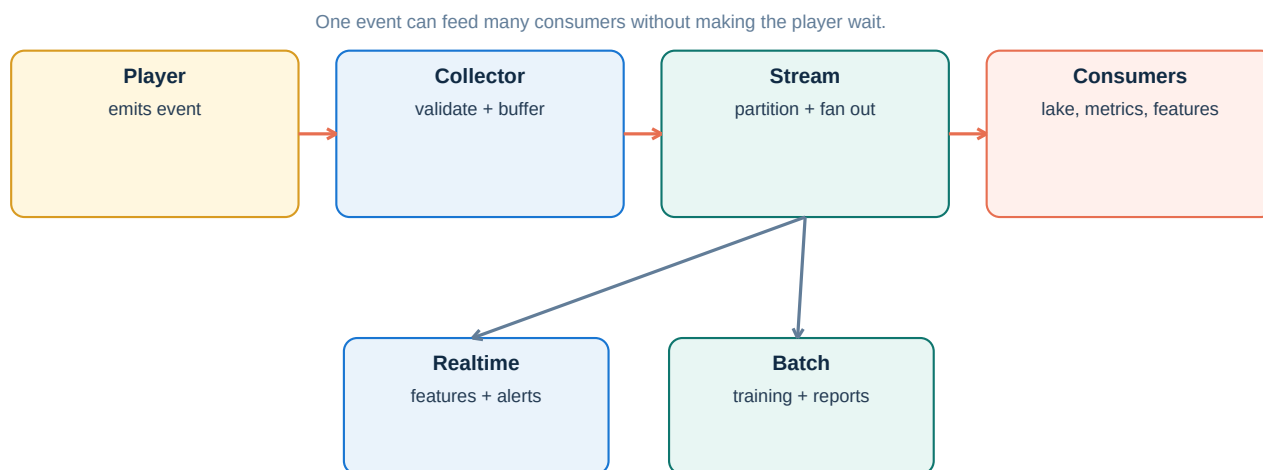
10. Sharding or partitioning

Sharding splits a large dataset across machines using a partition key. Good keys spread load evenly and keep common queries simple. Bad keys create hotspots.

Choice	Benefit	Risk
User ID	User data stays together; easy profile lookup.	One very popular user can still be hot, but usually manageable.
Content ID	Good for title-level data and catalog reads.	A blockbuster title can create a hot partition.
Time	Efficient for recent events and time-range scans.	The newest partition can receive almost all writes.

11. Queue, stream, and asynchronous work

A **queue** buffers work so a producer does not need to wait for a consumer. A **stream** is an ordered, append-only flow of events that many consumers can read. Use asynchronous work when it is not required to finish before the user gets a response.



Example: The video player sends a watch-completed event. The player should not wait for training data, analytics, and dashboards to finish. It publishes the event and continues.

12. CDN and object storage

Object storage holds large immutable files. A CDN caches those files near users. This is why a video or image can be served without making every request hit your application servers.

13. Rate limiting and backpressure

Rate limiting controls how much traffic a client can send. **Backpressure** makes producers slow down when consumers cannot keep up. Both protect the system from overload.

Technique	Simple idea	Example
Token bucket	Tokens refill over time; each request spends a token.	Allow 100 API calls per minute with short bursts.
Leaky bucket	Requests leave at a steady rate.	Smooth traffic sent to a slow downstream service.
Circuit breaker	Stop calling a failing dependency for a short period.	Do not keep hammering a down ranking service.
Timeout	Stop waiting after a bounded time.	Ranking has 80 ms; otherwise use fallback.

Part 5 - Data contracts, reliability, and observability

A system is easier to scale when its boundaries are explicit.

14. API design

An API is a contract between callers and the service. Keep it small, name fields clearly, define pagination, and make repeated requests safe when possible.

API concern	Plain-language question	Example
Pagination	How does the client get the next page?	cursor=eyJpZCI6MTIz; do not use an unstable offset for huge feeds.
Idempotency	Can a retry cause the action twice?	A payment request uses an idempotency key.
Versioning	How can old clients keep working?	v1 and v2, or additive fields with backward compatibility.
Partial failure	What if one dependency is unavailable?	Return cached recommendations and mark the response as fallback.

15. Idempotency, retries, and exactly-once myths

Retries help with temporary failures, but a retry can repeat a write. An idempotency key lets the server recognize that two requests are the same logical operation. "Exactly once" is rarely a free property across distributed systems; design for at-least-once delivery plus deduplication when needed.

Example

A client times out after creating a playlist. It retries with the same request ID. The service returns the original playlist instead of creating a duplicate.

16. Observability

Observability answers: what is happening, where, and why? Use three signals:

Signal	What it tells you	Recommendation example
Metrics	Numbers over time.	p50/p95/p99 latency, error rate, cache hit rate, empty-result rate.
Logs	Detailed events for a request or job.	Model version, fallback reason, and request ID.
Traces	The path and time spent across services.	Gateway -> features -> candidates -> ranker.

17. Security basics

Use authentication to identify callers and authorization to decide what they may do. Encrypt data in transit and at rest. Minimize personal data, restrict access, rotate secrets, validate input, and log security-relevant actions without leaking sensitive values.

Interview sentence

"I will make the request traceable with a correlation ID, put timeouts on every network hop, cap retries, and expose a safe fallback. For data, I will separate access control from storage and encrypt sensitive fields."

Part 6 - Capacity estimation without fear

The goal is not perfect arithmetic. The goal is to expose bottlenecks.

18. The four estimates to practice

Estimate	Formula	Why it matters
Average requests per second	daily requests / 86,400	Sizes the normal serving layer.
Peak requests per second	average RPS x peak factor	Sizes for traffic spikes.
Storage	events per day x bytes per event x retention	Chooses storage and partitioning.
Bandwidth	requests per second x payload bytes	Checks network and CDN needs.

19. Worked example: viewer events

Assume a Prime-like service has 20 million daily active viewers. Each viewer creates 30 events per day: play, pause, seek, finish, skip, and so on. That is 600 million events per day.

Step	Calculation	Approximate answer
Average event rate	600,000,000 / 86,400	6,944 events/sec
Peak event rate	6,944 x 5	34,720 events/sec
Raw event storage/day	600,000,000 x 500 bytes	300 GB/day
Raw storage/year	300 GB x 365	about 110 TB/year before compression and copies

If each event is 500 bytes, the event collector must absorb about 35,000 events per second at peak. That suggests buffering, partitioned streams, batch storage, and retention policies. It does not mean the database must synchronously write every event before the player continues.

Memory hook: average is for planning; peak is for survival

Always say the peak factor out loud. In interviews, a simple 3x to 10x assumption is more useful than silent arithmetic.

20. Latency budgets

If the total p99 goal is 200 ms, do not spend the full 200 ms in one downstream call. Reserve time for network hops, serialization, queueing, and a fallback. A rough budget could be: gateway 20 ms, feature retrieval 40 ms, candidate generation 30 ms, ranking 60 ms, response 20 ms, buffer 30 ms.

21. Common estimation mistakes

- Confusing daily active users with requests per second.
- Using average traffic when the system is sized for peak traffic.
- Ignoring response size and network bandwidth.
- Forgetting replicas, backups, indexes, and retention when estimating storage.
- Treating every event as a synchronous database write.

Part 7 - Trade-offs and mini designs

A strong answer knows what it is giving up.

22. Trade-off pairs worth remembering

If you optimize for...	You may pay with...	Typical decision
Low latency	Less freshness or more cache cost.	Cache popular lists; accept a short TTL.
Strong consistency	More coordination and sometimes higher latency.	Use for money or permissions.
High availability	Potentially stale or partial data.	Serve cached recommendations if ranking is down.
Simple operations	Less specialized performance.	Start with one service and one database.
Maximum throughput	More partitions and operational complexity.	Use streams and batch consumers.
Personalization quality	More data, features, and model complexity.	Add a ranker after a popularity baseline works.

23. Mini design: URL shortener

Requirement: turn a long URL into a short code and redirect quickly. **Simple design:** API -> ID generator -> key-value store. The short code is the key; the long URL is the value. Cache hot codes because reads are much more frequent than writes. Use a write-safe ID strategy so two writers do not generate the same code.

Trade-off: a random code needs collision checking; a sequence-based code is simple but may reveal volume. A distributed ID generator avoids one central bottleneck.

24. Mini design: notification service

Requirement: send email, SMS, and push notifications reliably. **Simple design:** producer -> durable queue -> channel workers -> provider adapters. Store delivery status and retry transient failures with backoff. Use an idempotency key so a retry does not send the same message twice.

Trade-off: asynchronous delivery improves resilience, but the caller must accept that delivery is not immediate. A dead-letter queue holds messages that keep failing.

25. Failure checklist

Question	Design response
What if a service is slow?	Timeout, bounded retry, circuit breaker, fallback.
What if a machine dies?	Load balancer removes it; replicated data remains.
What if one partition is hot?	Better key, sharding, caching, or hot-key replication.
What if a message is duplicated?	Idempotent consumer and deduplication key.
What if a region is down?	Multi-region failover if the business requires it; define data-loss and recovery targets.
What if data is stale?	TTL, version checks, refresh, or an explicit freshness SLO.

Part 8 - Capstone: a Prime-like recommendation system

A full walkthrough from the product question to the data science and ML components.

Scope note

We are designing a hypothetical streaming recommendation system similar to the experience a viewer may expect from a Prime-like service. This is an interview architecture, not a description of Amazon's private production systems.

26. The product problem

When a viewer opens the home page, the catalog may contain millions of titles. We cannot show everything, and we cannot run an expensive model over every title. The system must quickly choose a small set of useful candidates, rank them for this viewer, apply product rules, and return rows such as "Because you watched...", "Continue watching", and "Trending now".

27. Why do we need a recommendation system?

- Reduce the time a viewer spends searching for something to watch.
- Increase the chance that a viewer starts and completes a title.
- Balance personal taste with freshness, discovery, and business rules.
- Make the catalog feel smaller and easier to navigate.
- Learn from feedback instead of hand-curating every row.

28. Define success before choosing a model

Goal	Metric	Important caution
Viewer usefulness	play-start rate, watch time, completion rate	Do not optimize clicks alone; clickbait can hurt long-term satisfaction.
Freshness	time from new event to feature update	A user who just finished a series should see that reflected soon.
System quality	p95/p99 latency, error rate, fallback rate	A perfect model is useless if the home page is slow.
Catalog health	coverage, diversity, new-title exposure	Do not show the same narrow set to everyone.
Business health	retention, subscription conversion, complaints	Keep policy and user trust in the loop.

Part 9 - Recommendation system requirements and assumptions

State the problem so the architecture has something concrete to satisfy.

29. Functional requirements

- Return several recommendation rows for a logged-in viewer.
- Support anonymous or new viewers with non-personalized recommendations.
- Use recent behavior such as recent plays, skips, searches, and likes.
- Support different surfaces: home page, title detail page, and next-up screen.
- Exclude unavailable, blocked, already-watched, or age-inappropriate titles.
- Record impressions and actions so the system can learn.

30. Non-functional requirements

Requirement	Illustrative assumption for the interview
Scale	20 million daily active viewers; 2 million concurrent sessions at peak.
Latency	p95 home-page recommendation response under 200 ms; p99 under 400 ms.
Availability	99.9% or higher for serving; a useful fallback is acceptable.
Freshness	Recent events influence the viewer within a few minutes; some batch features refresh daily.
Data retention	Raw events kept for analysis and training according to privacy policy; aggregates kept longer.
Consistency	Catalog availability and permissions are strong; recommendations can be eventually consistent.

31. Non-goals for the first version

- A globally perfect model for every language and device from day one.
- Real-time retraining after every single event.
- A giant microservice fleet before the access patterns are proven.
- Predicting every possible user action; begin with play-start or watch-time objectives.

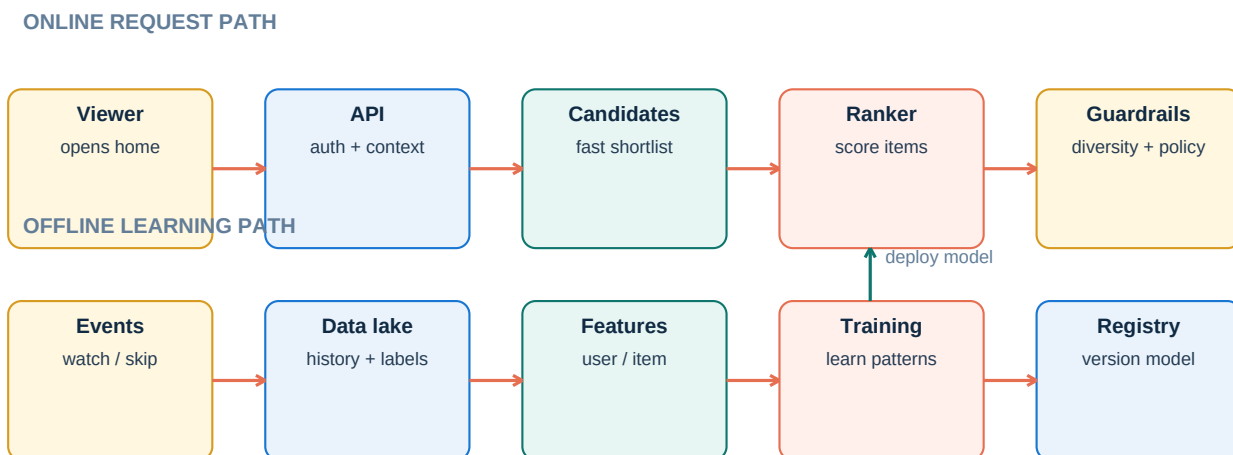
Simple design first

Version 1 can be a popularity and genre-based service with a cache. Once it is measurable and reliable, add personalized candidates, then a learned ranker. This staged path is easier to debug and defend in an interview.

Part 10 - High-level architecture

Separate the fast online path from the slower learning path.

32. The big picture



The upper lane answers the viewer now. The lower lane learns from past behavior and supplies models and features to the upper lane. This separation protects the viewer request from slow analytics and training jobs.

33. Main components

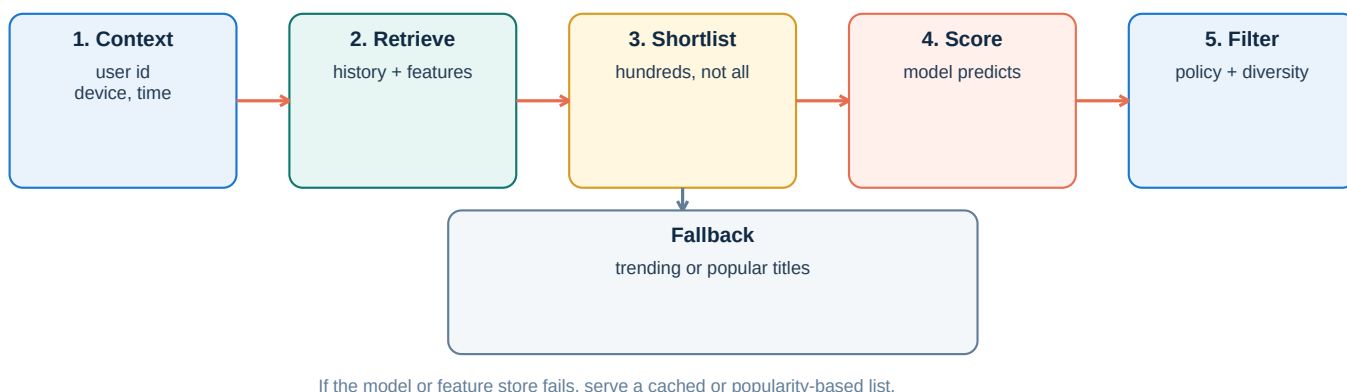
Component	Responsibility	Why it exists
Recommendation API	Accepts viewer and surface context; returns rows.	Stable product contract.
Feature store	Serves user, item, and context features.	Fast, consistent model inputs.
Candidate generators	Produce a few hundred likely titles.	Avoid scoring millions of titles.
Ranker	Scores candidates for this viewer and context.	Improves ordering and relevance.
Policy / re-ranker	Applies availability, safety, diversity, and business constraints.	Prevents a model from making the final decision alone.
Event pipeline	Collects plays, skips, impressions, and other signals.	Creates feedback and training data.
Model registry	Stores approved model versions and metadata.	Enables controlled deployment and rollback.

Part 11 - Request-time flow: from open screen to ranked rows

Trace one request end to end.

34. Step-by-step serving path

REQUEST-TIME DETAIL: return a useful answer even when one dependency is slow.



- **1. Context:** read viewer ID, device, language, country, time, surface, and experiment assignment.
- **2. Retrieve features:** get recent history, long-term preferences, item availability, and model features from fast stores.
- **3. Generate candidates:** fetch candidates from several sources, such as collaborative filtering, similar titles, trending, and continue watching.
- **4. Score:** the ranker predicts a useful outcome, for example probability of starting or expected watch time.
- **5. Filter and organize:** remove unavailable or unsafe titles, avoid duplicates, add diversity, and build labeled rows.
- **6. Respond:** return titles, images, explanations if supported, model metadata, and a request ID.

Why candidate generation comes first

Ranking every catalog title would be too slow and expensive. Candidate generation reduces millions of titles to a few hundred. The ranker then spends more compute where it matters.

35. Example API

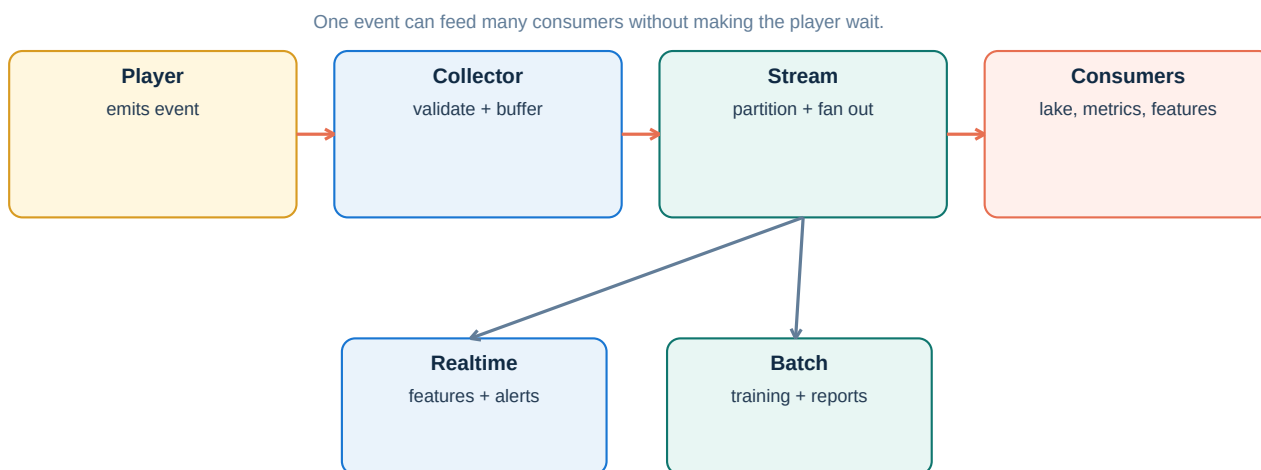
Request	GET /v1/recommendations?surface=home&limit=20
Inputs	viewer_id, device, locale, time, experiment_id, content_policy
Response	rows[[row_id, title_ids[]], reason, expires_at[]], request_id, model_version, fallback_used
Timeout policy	If features or ranking exceed their budget, return cached or popular rows.

Part 12 - Event collection and data flow

The model learns only if the system records what happened, including what was shown but ignored.

36. Events to collect

Event	Useful fields	Why it matters
Impression	viewer, title, row, position, timestamp, model version	Without impressions, a skip is hard to interpret.
Play start	viewer, title, position, timestamp, device	Core positive signal.
Watch progress	title, seconds watched, percent complete	Distinguishes a click from real engagement.
Skip / dislike	title, reason if available, timestamp	Negative or corrective feedback.
Search / browse	query, filters, titles opened	Shows intent even when no play starts.
Feedback	rating, add-to-list, share, report	Explicit signals and safety feedback.



37. Event design details

- **Stable event ID:** helps deduplicate retries.
- **Event time and ingestion time:** separates when the viewer acted from when the backend received it.
- **Schema version:** lets consumers evolve without guessing field meaning.
- **Privacy controls:** minimize, redact, encrypt, and honor retention and deletion requirements.
- **At-least-once is normal:** consumers should be idempotent because a message may arrive twice.

A subtle but important point

Log both positive actions and the opportunities that were not chosen. If a title was never shown, a missing play does not mean the viewer disliked it.

Part 13 - Candidate generation

Use several inexpensive sources; no single method sees every useful title.

38. Candidate sources

Source	Simple explanation	Strength	Weakness
Continue watching	Titles the viewer started but did not finish.	Highly relevant and easy to explain.	Small coverage; not discovery.
Popular / trending	Titles popular in the viewer's region or cohort.	Strong cold-start fallback.	Same list for many people.
Content similarity	Titles with similar genre, cast, language, or embeddings.	Works for new titles with metadata.	May repeat the same taste.
Collaborative filtering	Users with similar behavior liked these titles.	Captures taste beyond metadata.	Needs enough interaction history.
Sequence model	Uses order and timing of recent actions.	Understands short-term intent.	More complex and sensitive to drift.

39. Combining candidates

Fetch a limited number from each source, merge by title ID, remove unavailable items, and keep a source label for debugging. For example: 50 collaborative-filtering titles + 30 similar-title titles + 20 trending titles + 10 continue-watching titles.

Memory hook: retrieve wide, rank narrow

Candidate generation is about recall: do not miss good options. Ranking is about precision: put the best options first.

40. Candidate store

Precompute many candidate lists offline and store them by viewer, cohort, item, or region. At request time, read a few lists and combine them. This keeps the online path fast and predictable.

41. Cold-start cases

Cold start	Fallback
New viewer	Regional popularity, editorial rows, language/device context, and a few onboarding choices.
New title	Metadata similarity, editorial signals, early watch signals, and exploration budget.
New region	Local catalog constraints, regional trends, and policy-aware defaults.
Sparse history	Blend personal signals with cohort and global signals.

Part 14 - Ranking and re-ranking

The ranker chooses order; policy makes the final result safe and useful.

42. Ranking objective

The ranker estimates a score for each candidate. One simple score could combine predicted play probability, expected watch time, freshness, and a penalty for repetition. The exact formula depends on the product goal.

Illustrative score: $0.5 \times \text{play probability} + 0.3 \times \text{expected minutes watched} + 0.1 \times \text{freshness} + 0.1 \times \text{discovery value} - \text{repetition penalty}$.

The weights are not magic constants. They are a product decision to be tested with offline evaluation and online experiments.

43. Features

Feature group	Examples
Viewer	recent genres, language, completion rate, watch-time distribution, subscription tier if allowed.
Title	genre, cast, language, release age, runtime, maturity rating, quality signals.
Interaction	viewer-title count, last interaction time, seconds watched, skip history.
Context	device, time of day, weekday, network quality, surface, experiment.
System	title availability, regional rights, current popularity, model freshness.

44. Re-ranking rules

- Remove titles that are unavailable in the viewer's region or plan.
- Apply age, language, and parental-control filters before showing the title.
- Limit repeated titles across rows.
- Add diversity across genre, language, creators, and release period.
- Reserve a small exploration slot for new or uncertain titles.
- Respect editorial, legal, and business constraints with explicit policy code.

Why not let the ML model decide everything?

A model optimizes the data and objective it sees. Policy changes faster than training data, and some constraints are non-negotiable. Keep them explicit, testable, and auditable.

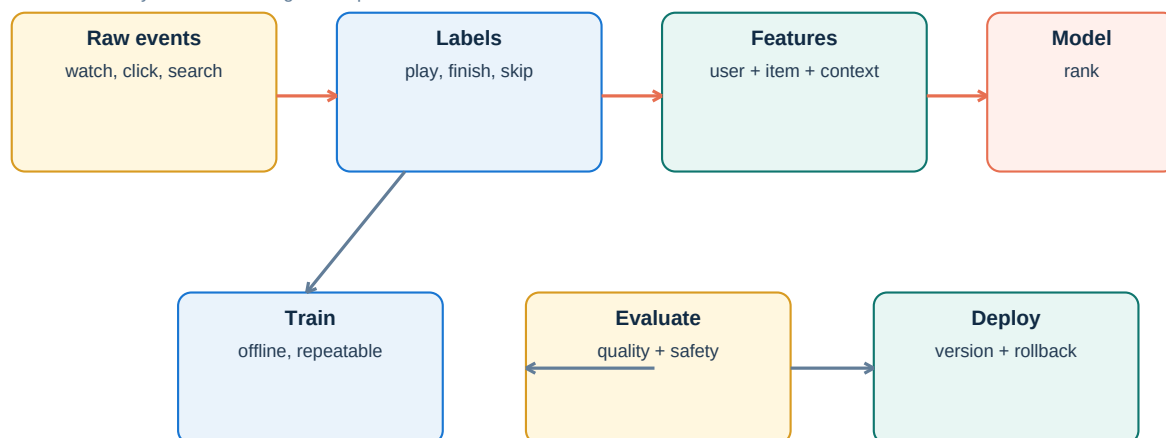
Part 15 - Data science, AI, and ML components

This is the end-to-end learning system behind the serving boxes.

45. What the data science team owns

Area	Plain-language job
Problem framing	Choose the target: play-start, watch time, completion, retention, or a multi-objective combination.
Data and labels	Define what counts as positive, negative, delayed, censored, or missing feedback.
Features	Create useful signals without leaking future information into training.
Modeling	Choose baseline, candidate-generation, ranking, and sequence approaches.
Evaluation	Measure offline quality, bias, coverage, freshness, and online business impact.
Experimentation	Run safe A/B tests with guardrails and a rollback plan.

The model is only one box in a larger data product.



46. A practical model ladder

Stage	Model	Why use it
0	Popular by region / cohort	Baseline, fallback, and cold-start system.
1	Content similarity	Useful for new titles and explainable recommendations.
2	Collaborative filtering	Learns behavior-based similarity.
3	Gradient-boosted or neural ranker	Combines many features for better ordering.
4	Sequence / multi-task model	Captures recent intent and several objectives.

Part 16 - Training pipeline and feature store

Training should be repeatable, versioned, and safe to roll back.

47. Training pipeline

- **Ingest:** read validated events and catalog snapshots.
- **Join:** connect impressions to later actions using stable IDs and time windows.
- **Build labels:** for example, play within 24 hours; avoid treating every non-play as a dislike.
- **Build features:** compute user, item, interaction, and context features as-of the prediction time.
- **Split by time:** train on older data and validate on newer data to mimic production.
- **Train and evaluate:** log code version, data version, model parameters, and metrics.
- **Register:** approve the model, publish artifacts, and define rollback criteria.
- **Deploy:** canary or A/B test before full traffic.

48. Feature store in simple terms

A feature store is a governed place to compute and serve model inputs. Its main value is consistency: the feature used in training should mean the same thing when computed online.

Feature	Offline example	Online example
recent_genre_counts	Count genres watched in the last 30 days.	Read a precomputed vector in milliseconds.
title_popularity_1h	Aggregate plays over a historical hour window.	Update from the event stream every few minutes.
viewer_title_seconds	Total seconds watched for a viewer-title pair.	Read from a low-latency key-value store.
completion_rate	Completed plays / started plays over a time window.	Serve a cached, versioned value.

Data leakage warning

Do not let a training feature use information that would not exist at prediction time. If the label is "play tomorrow," a feature calculated from tomorrow's behavior leaks the answer.

49. Model serving choices

- Precompute recommendations when freshness needs are modest.
- Use online scoring when context changes the ranking significantly.
- Use a hybrid: precompute candidates, score and filter online.
- Keep model versions and feature schemas compatible; reject invalid inputs safely.

Part 17 - Online serving, reliability, and cost

A recommendation system is a product service with an ML component, not just a model endpoint.

50. Latency budget example

Stage	Budget	Protection
API gateway and auth	20 ms	Connection pooling and local token validation.
Feature retrieval	40 ms	Batch reads, cache, and timeout.
Candidate retrieval	30 ms	Precomputed lists and parallel fetches.
Ranking	60 ms	Small candidate set, bounded model complexity.
Policy and response	20 ms	In-memory filters and compact payload.
Safety buffer	30 ms	Fallback before the total deadline.

51. Failure modes and fallbacks

Failure	Fallback	User impact
Feature store slow	Use cached features or cohort-level features.	Less personal but still useful.
Candidate service down	Use cached candidate lists or popularity.	Less fresh, still navigable.
Ranker timeout	Return candidates in source order or cached ranked rows.	Lower quality, acceptable latency.
Catalog dependency down	Use last-known availability with short TTL, or suppress uncertain titles.	Fewer recommendations but safer.
Bad model detected	Rollback to previous version or baseline.	Quality returns to a known state.

52. Cost controls

- Cache popular and precomputed lists to reduce repeated model work.
- Batch expensive feature calculations instead of recomputing per request.
- Use smaller models online; reserve large models for offline candidate generation or re-ranking.
- Keep raw events and derived aggregates under explicit retention policies.
- Scale stream consumers independently from the viewer-serving path.

Interview sentence

"I am optimizing the online path for predictable latency. The ML pipeline can be more expensive offline, but the home-page request needs bounded work, parallel reads, timeouts, and a useful fallback."

Part 18 - Evaluation, experimentation, and feedback loops

Measure what the viewer experiences, not just what the model predicts offline.

53. Offline metrics

Metric	Meaning	Caution
Precision@K	How many of the top K items were relevant.	Needs a trustworthy relevance label.
Recall@K	How much of the relevant set appeared in top K.	Can favor broad lists without good ordering.
NDCG@K	Rewards putting more relevant items near the top.	Still depends on labels and logging quality.
AUC / log loss	How well predictions rank or calibrate outcomes.	Good model metric, not the final product metric.
Coverage	How much of the catalog receives exposure.	Protects against showing only blockbusters.
Diversity	How varied the results are.	Must be balanced with relevance.

54. Online A/B testing

Randomly assign viewers to a control and a treatment. Keep the assignment stable, define the experiment window, and monitor primary metrics plus guardrails such as latency, error rate, complaints, retention, and catalog coverage.

Do not ship a model because one metric moved. Check statistical confidence, segment behavior, novelty effects, and whether the result persists.

55. Feedback loop risks

- **Popularity bias:** popular titles get shown more, collect more plays, and become even more popular.
- **Position bias:** viewers click the first row because it is first, not because it is best.
- **Filter bubbles:** narrow recommendations reduce discovery and may harm long-term satisfaction.
- **Delayed outcomes:** a viewer may start a title today and finish it days later.
- **Missing-not-at-random data:** you observe actions only for what the system chose to show.

Healthy loop

Log impressions -> define labels carefully -> train with time-correct features -> evaluate offline -> A/B test with guardrails -> monitor drift and user impact -> retrain or roll back.

Part 19 - Privacy, safety, and governance

Personalization is powerful, so the boundaries must be explicit.

56. Protecting viewer data

- Collect only signals needed for the recommendation objective.
- Separate identity data from behavioral data where possible; use controlled identifiers.
- Encrypt data in transit and at rest; restrict access by role and purpose.
- Honor deletion, retention, consent, parental-control, and regional requirements.
- Avoid exposing sensitive history in explanations, logs, or analytics dashboards.
- Audit model features and outputs for unfair or unsafe behavior.

57. Content and policy layer

Policy is not just a model feature. Rights, age rating, geography, subscription plan, takedown status, and safety decisions should be validated close to serving. If uncertain, suppress the title or use a safe known state.

58. Governance checklist

Question	Expected answer
Which model generated this row?	Store model version and experiment assignment with the response.
Which features influenced it?	Keep feature definitions and lineage.
Can we reproduce the decision?	Retain request context and candidate IDs under privacy controls.
Can we stop a bad rollout?	Canary, monitoring, alert, and rollback to baseline.
Can the viewer control personalization?	Provide settings, feedback, and clear policy behavior.

Part 20 - A polished interview answer you can say aloud

Use this as a rehearsal script, then adapt it to the interviewer's follow-up.

59. Opening

"I will design a Prime-like streaming recommendation service. I will optimize for fast, useful, and safe recommendations, while accepting eventual consistency for personalization. I will first clarify requirements and assumptions, then show the online serving path and the offline data/ML path."

60. Requirements and scale

"Let us assume 20 million daily active viewers, a peak of 2 million concurrent sessions, a p95 recommendation latency below 200 ms, and a few-minute freshness target for recent behavior. The catalog and permissions need strong correctness; recommendations may be slightly stale."

61. Architecture

"The request enters through an API gateway. The recommendation service reads context and features, retrieves candidates from multiple sources, scores them with a ranker, applies availability and diversity rules, and returns rows. I will cache common lists and keep a popularity fallback. Separately, an event pipeline sends impressions and actions to streaming consumers and a data lake. A training pipeline builds time-correct features, trains a versioned model, evaluates it, and deploys it through a registry."

62. Reliability

"Every network hop gets a timeout. Retries are bounded. If feature retrieval or ranking is slow, the service returns cached or popular recommendations. We track p95 and p99 latency, error rate, fallback rate, cache hit rate, coverage, freshness, and model quality."

63. Trade-off close

"I am choosing a hybrid design because precomputation gives predictable latency, while online filtering and ranking use current context. The cost is more data and model-serving complexity, so I would start with a popularity baseline, then add personalization only after the baseline is measurable and reliable."

If the interviewer asks for more depth

Pick one branch: feature-store consistency, candidate generation, model objective, multi-region design, cold start, experimentation, or privacy. Do not redraw the entire system every time.

Part 21 - Final one-page memory sheet

Review this before the interview and use it to reconstruct the details.

64. The universal system-design checklist

Question	Your answer should cover
Who uses it?	Users, clients, regions, authentication, permissions.
What do they do?	Core reads, writes, events, and product boundaries.
How much traffic?	Average and peak RPS, payload size, storage, bandwidth.
What is the hot path?	Client -> edge -> gateway -> service -> cache/database.
What is asynchronous?	Events, analytics, notifications, training, heavy jobs.
Where is state?	Database, cache, object store, stream, feature store.
What fails?	Timeout, retry, circuit breaker, fallback, duplicate, hot key, region loss.
What do we measure?	Latency, errors, availability, freshness, correctness, cost, business metric.
What are the trade-offs?	Consistency versus availability, freshness versus latency, quality versus cost.

65. Recommendation-system checklist

Layer	Remember
Product	What should improve for the viewer?
Events	Impressions plus actions, stable IDs, schema and privacy.
Candidates	Continue watching, popular, content, collaborative, sequence.
Ranking	User + item + interaction + context features.
Policy	Availability, safety, rights, diversity, exploration.
Serving	Fast features, bounded candidate set, cache, fallback.
Learning	Time-correct labels, train, evaluate, register, deploy, monitor.
Success	Watch behavior, satisfaction, freshness, latency, coverage, safety.

Last memory hook

A good system is not a diagram full of boxes. It is a set of clear decisions about requirements, data, scale, failure, and trade-offs.

End of guide